

Comparative Study of Iterative Methods for Inverse Kinematics of Redundant Serial Robots with Increasing Degrees of Freedom

Luis A. Orbegoso Moreno*, D. Valverde Ramírez, M. Pasco Sánchez and L. Ruiz Rodríguez

Department of Mechatronic Engineering

National University of Trujillo

Trujillo, Peru

lorbegoso@unitru.edu.pe, dvalverde@unitru.edu.pe, jpasco@unitru.edu.pe, jlruiz@unitru.edu.pe

Abstract—This paper presents a comparative study of five state-of-the-art iterative methods - Particle Swarm Optimization, Quantum Particle Swarm Optimization (PSO), Genetic Algorithms (GA), Damped Least Squares (DLS), and Forward and Backward Reaching Inverse Kinematics (FABRIK) - used to solve the challenging inverse kinematics problem in robots with increasing degrees of freedom in their kinematic chains. The analysis includes 7, 9, 11, 13, and 15 degrees of freedom redundant serial robots with hinge and pivot joints. Performance indicators, such as execution time, iteration count, and final error, were evaluated for each method across 500 randomly generated target poses for the five robotic chains, providing valuable insights into the algorithms' behavior as the degrees of freedom in the kinematic chains increase.

Keywords—inverse kinematics, PSO, QPSO, GA, DLS, FABRIK, redundant serial robots

I. INTRODUCTION

The inverse kinematics (IK) problem involves finding a set of joint values for the kinematic chain of a body to reach a goal in the task space, and the methods used to solve this problem can be mainly classified into analytical, data-driven or learning, numerical, and heuristic methods [1]. Now, a kinematic chain is considered redundant when it has more degrees of freedom (DoF) than is needed to perform a specific task or reach a certain position in the workspace; in other words, there are more linkages in the chain than are required to achieve a desired end-effector configuration.

Starting with analytical methods, these rely on symbolic expressions to find all the solutions of IK, using constraints or assumptions to choose a single solution among all the possibilities [2]. By way of example, in works [3] and [4], different analytical methods were proposed to find the IK of robots with 6 and 7 DoF respectively. The advantage of this method is that it does not require iterations or data learning to give exact solutions, however the main disadvantage is that its solutions are limited to restricted cases with low DoF.

Thus, data-based methods take advantage of the learning capabilities of neural networks to arrive at solutions to inverse kinematics [5]. In the article [6] an inverse kinematic solver for a 3 DoF robotic arm based on an artificial neural network (ANN) with three hidden layers is presented, in which GA were

used to reduce manual adjustments of the network, thus achieving a precision of less than 0.27 mm for each axis. Other work presented in [7], developed a deep reinforcement learning (RL) approach to solve the IK problem in a 7 DoF robotic manipulator, using the product of exponentials (PoE) and Deep Q-Network (DQN) to generate joint spatial trajectories. Now, the advantage of these methods is that they also do not require iterations to find a solution to the IK and that they are also capable of performing much more complex tasks; however, everything depends on the training and the amount of data available, on which the learning time depends and avoid falling into overfitting.

On the other hand, numerical methods focus on minimizing a cost function expressed by the error between the end effector and the target of the kinematic chain [8]. Usually, techniques such as the Newton method and its variants are used, which require the inversion of the Jacobian matrix of this cost function, which is not always invertible, which is why alternative approaches have been developed such as the Transposed Jacobian, Singular Value Decomposition (SVD-DLS), Damped Least Squares (DLS) and the incorporation of joint constraints as presented in [9] to find solutions for the IK problem. These methods have generally been used using homogeneous transformation matrices, however in [10] it was shown that the use of dual quaternions as a tool for the representation of spatial coordinates can significantly improve convergence. The advantage of these methods is that they can be used in almost any IK problem; however, the main disadvantage is that these methods need the derivatives and more specifically the Jacobian, in addition to the fact that they require iterations to find solutions, which brings a high computational cost [11].

Now, heuristic methods are problem-solving techniques that are based on the search and exploration of the solution space without guarantees of finding the optimal solution, but they are efficient in finding satisfactory solutions or close to the optimal solution in complex problems. The PSO is a heuristic method described for the first time in [12] whose paradigm is based on the behavior of swarms of insects and has been used to solve problems of different dimensionality among which is IK as shown in the work [13] where it was used for a KUKA KR 6 R900 robot, also in the works [14,15] the PSO was also

used to plan trajectories based on the solutions of the IK for redundant kinematics robots. Later, in [16] the quantum behavior of the particles to the PSO was introduced, giving rise to the Quantum-behaved PSO (QPSO) which turns out to outperform its predecessor for certain problems, as evidenced in [17] where it was validated that for a 7 DoF robot the QPSO turned out to have better efficiency than the firefly algorithm (FA), artificial bee colony (ABC) and PSO.

Genetic algorithms (GA) on the other hand, are heuristic methods based on the paradigm of biological evolution, natural selection, reproduction, and the survival of the fittest to solve optimization and search problems. Thus, in the work [18] a solution of the IK problem of robotic manipulators using GA was presented, formulating the problem as an optimization to minimize the cumulative deviation of the trajectory in the absence of obstacles in the workspace. another work presented in [19] proposed a solution to address IK problems of multiple fingers connected to the same wrist.

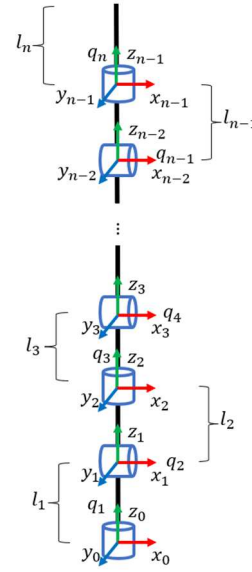
Finally, the FABRIK method first introduced in [1] is an IK metaheuristic method characterized by its simplicity and fast convergence using only vector operations and having applications for computer graphics, presenting a better performance than a previous method in this field known as cyclic coordinate descent (CCD). Despite these advantages, FABRIK was initially created to work with ball joints that are far from the typical DoF joints of robots, so works such as [20, 21, 22, 23, 24] have focused on adapting this algorithm to robotic kinematic chains, as well as for mobile robotic platforms [25] and task space constraints [26].

Now, the objective of the present work is to compare in indicators of time, number of iterations and final error to five iterative methods with different paradigms (PSO, QPSO, DLS, GA and FABRIK) used in the resolution of the IK, focusing on redundant kinematic chains of 7, 9, 11, 13 and 15 DoF, for which 500 random postures will be used for each of the kinematic chains to evaluate the performance in these 3 evaluation indicators. Next, this article is structured as follows: Section 2 addresses the kinematic parameters of the 5 kinematic chains used in the evaluation, as well as obtaining the random postures and the definitions of the algorithms to be compared; Section 3 deals with the results obtained in the evaluations of these algorithms with the respective data graph; and finally, Section 3 deals with the conclusions reached.

II. MATERIALS AND METHODS

A. Kinematic chains

In this work, the Screw Theory was used to model the direct kinematic system of redundant serial robots with 7, 9, 11, 13 and 15 DoF, consisting of hinge and pivot joints, and translations along the z-axis and rotations around the z- and x-axes were performed for the pivot and hinge joints, respectively, as indicated in Fig 1.



1. Model of the kinematic chain for an “n” DoF robot

The sequence of these rotations and translations of the kinematic chain of the test modeled robot is shown in Table 1, which also shows the joint limits assumed for the joints of the n DoF robots, as well as the lengths of the links.

I. PARAMETERS OF FORWARD KINEMATICS USING THE SCREW THEORY

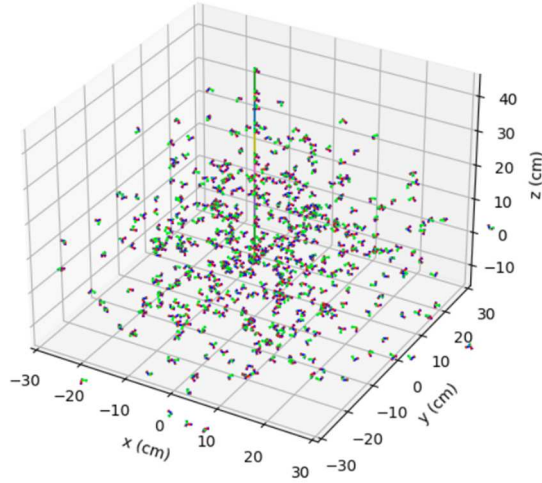
Link Number	Rotation		Translation	
	Axis	Angle (rad)	Axis	Value (cm)
1	[0, 0, 1]	$[-\pi, \pi]$	[0, 0, l_1]	2
2	[1, 0, 0]	$[-\frac{5}{6}\pi, \frac{5}{6}\pi]$	[0, 0, l_2]	2
3	[0, 0, 1]	$[-\pi, \pi]$	[0, 0, l_3]	7
4	[1, 0, 0]	$[-\frac{5}{6}\pi, \frac{5}{6}\pi]$	[0, 0, l_4]	2
5	[0, 0, 1]	$[-\pi, \pi]$	[0, 0, l_5]	7
⋮	⋮	⋮	⋮	⋮
n-1	[1, 0, 0]	$[-\frac{5}{6}\pi, \frac{5}{6}\pi]$	[0, 0, l_{n-1}]	2
n	[0, 0, 1]	$[-\pi, \pi]$	[0, 0, l_n]	6

It can be seen that there is a pattern from link 2, where for each hinge joint with limits $[-\frac{5}{6}\pi, \frac{5}{6}\pi]$ is followed by a translation of 2 cm and continues with a pivot joint with limits $[-\pi, \pi]$ followed by a translation of 7 cm, and so on until reaching link n, where the last translation in the z axis is 6 cm.

B. Generation of random poses

In order to determine the random postures within the task space of the robots that must be generated in order to make the comparisons between the intervening algorithms, this was done from random angles contained in the admissible intervals of each joint, to which, once obtained, the direct kinematics of each one of the robots was applied to find the random posture that is therefore admissible for the kinematic model of the robot. Now, taking [17] as a reference, where 100 random

points were considered to perform the evaluation and comparison of algorithms, for this work 500 random postures were considered for each of the 5 robots, as shown in Fig 2.



2. Random pose cloud for the 13 DoF robot

C. Iterative algorithms

In this work, the PSO, QPSO, GA, DLS and FABRIK methods were considered to compare the time, number of iterations and final error. Now, the equation of the error used to optimize by these five algorithms is shown in the following expression.

$$E = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2 + \Delta O_{1,2}^2 + \Delta O_{0,2}^2 + \Delta O_{1,0}^2} \quad (1)$$

In equation (1), being P_t the target position and P_e the actual end effector position, therefore the position error is $P_t - P_e = (\Delta x, \Delta y, \Delta z)$, where Δx is the error in the x-axis, Δy is the error in the y-axis and Δz is the error in the z-axis. On the other hand, being O_t the target orientation and O_e the actual end effector orientation, therefore the orientation error is $\Delta O = O_t O_e^T$, of which $\Delta O_{1,2} = O_{2,1} - O_{1,2}$, $\Delta O_{0,2} = O_{2,0} - O_{0,2}$ and $\Delta O_{1,0} = O_{0,1} - O_{1,0}$ are the alignment errors between the axes of one orientation with respect to the other using rotation matrices.

The algorithms, when starting each test to achieve a set objective, always started with the angles of the kinematic chain at zero, and after carrying out the respective iterations until reaching some answer, these angles were reset to zero to carry out the next test and all the internal parameters that the algorithms could have been reset. Also, the algorithms were programmed to stop once this error falls below the threshold value of 0.001 or when the maximum number of iterations of 1000 is reached. The specifications of each of these comparison algorithms are shown below.

1) *DLS*: The DLS method developed in this work used the Jacobian of the inverse kinematics performed with dual quaternions as indicated in [10], in addition to ensuring that the values of the joints are within the limits established in the kinematic configurations of the robots, the technique

suggested by [9] was used. Thus, the iterative equation to calculate the joint values of the robot is shown below.

$$\theta_i = \theta_{i-1} - (J^T J + I \lambda^2)^{-1} J^T E_D \quad (2)$$

Where E_D is the dual quaternion error expressed as the difference between the target dual quaternion and the actual end effector dual quaternion. Also, θ is a vector of the joint angles of the kinematic chain; J is the Jacobian of the forward kinematics and the damping factor if computed dynamically using the following equation.

$$\lambda = \frac{E_D \cdot J^T \cdot E_D}{|J \cdot J^T \cdot E_D|} \quad (3)$$

2) *PSO*: The equations that govern the behavior of each particle "i" in the PSO algorithm are shown below.

$$v_i = \omega v_{i-1} + c_1 r_1 (p_{l_{best}} - x_i) + c_2 r_2 (p_{g_{best}} - x_i) \quad (4)$$

$$x_i = x_{i-1} + v_i \quad (5)$$

Where x_i is the state of the particle "i", v_i is the velocity of the particle, ω is the inertial weight (for this work the value is between 0.4 and 0.9 with an initial value of 0.8), c_1 is the cognitive parameter (value 1 for this work), c_2 is the social parameter (value 2 for this work), r_1 and r_2 are random parameters, $p_{l_{best}}$ is the best local position and $p_{g_{best}}$ is the best global position. The number of particles used were 40, value based on work [27] where a number of particles between 20 and 60 were used for a 7 DoF robot.

3) *QPSO*: The equations that govern the behavior of each particle "i" in the QPSO algorithm are shown below.

$$x_i = q_i \pm \beta (m_{best} - x_{i-1}) \log\left(\frac{1}{u}\right) \quad (6)$$

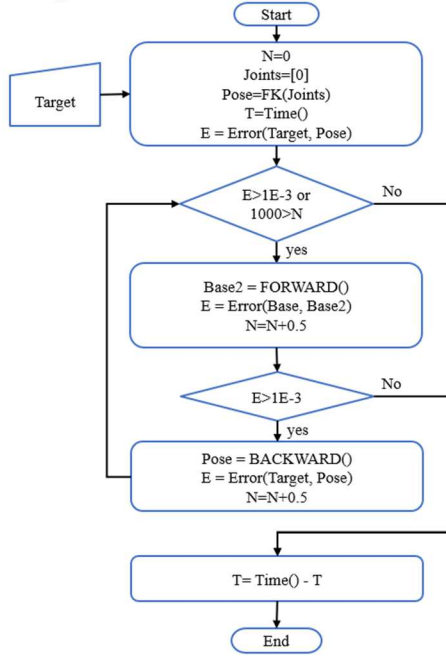
$$g_i = \varphi \cdot p_{best} + (1 - \varphi) g_{best} \quad (7)$$

$$m_{best} = \frac{\sum_{i=1}^M p_{best_i}}{M} \quad (8)$$

Where φ and u are random values between 0 and 1, if $u \geq 0.5$ the sign of equation (6) is positive, otherwise it's negative; β is the shrinkage-relaxation parameter (value of 0.5 for this work); and M in the number of particles (40 for this study).

4) *GA*: To define the parameters used in the GA used in the comparison of the present work, it was based on [19] and the Deap python library was used for the implementation. For the algorithm the population was 300, the number of genes where equal the DoF of the kinematic chains, the cross type was the Simulated Binary Crossover, the cross rate for this study was 0.8, the mutation type defined was Gaussian, the mutated genes percentage was 5% and the selection was Tournament-3.

5) *FABRIK*: Since the original *FABRIK* algorithm is not designed to work with one-degree-of-freedom robotic joints such as kinematic chains in this study, the adaptation of this algorithm presented in [24] was used, whose flowchart is shown in Fig 3.



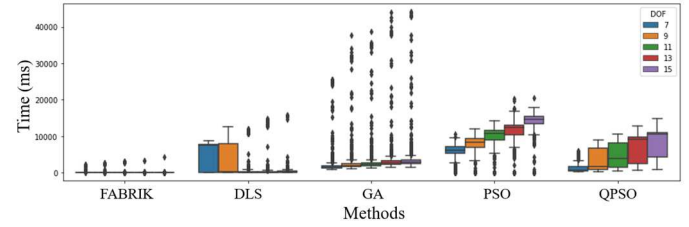
3. FABRIK algorithm adaptation flowchart

As observed in the previous figure, like the other algorithms, the first procedure is to initialize the "N" counter to zero, establish the robot's angles as a vector of zeros, calculate the current posture of the robot to calculate the first error, and make the initial measurement of time. Then, if it turns out that the measured error is less than the established threshold of 0.001 or if the counter exceeds the limit of 1000 iterations, then the loop breaks; Otherwise, the FORWARD stage of the *FABRIK* algorithm is continued (in this stage the kinematic chain is traveled from the end effector to the base of the robot, reconfiguring the links according to the planes of movement of the pivot or hinge joints) which contributes to the middle of the first iteration. Once this stage is finished, an error measurement is made again, but this time for the base of the robot; once again, if the error is less than the established threshold, the method is terminated, otherwise the BACKWARD stage continues, which does exactly the same as the FORWARD stage but starting from the base of the robot until reaching the final effector; and the loop is repeated until the termination condition is met; once the loop is finished the time difference is taken to see the time it took to reach a solution.

III. RESULTS

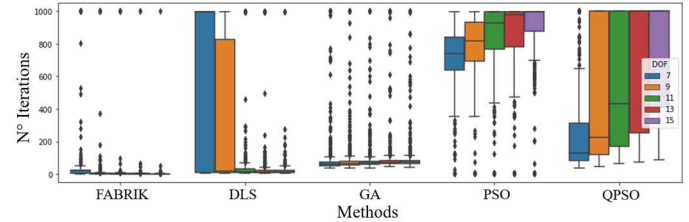
Fig 4 shows the dispersion of the time used according to the method and the DoF of the robot ordered ascendingly in each method using a distinctive color for each DoF as shown in the legend of the figure. It can be distinguished that, of all the algorithms, *FABRIK* is the one that takes the least time to carry

out its work, unlike the other methods such as GA, PSO and QPSO whose invested time increases as the DoF of the robot increase. However, in the case of the DLS method, for DoF of 11, 13 and 15 it is seen that the median time spent decreased compared to the median time in DoF of 7 and 9.



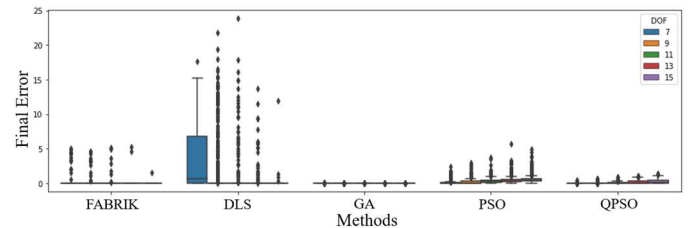
4. Box-and-whisker plot of Time vs methods and DoF

On the other hand, Fig 5 shows the dispersion of the number of iterations used according to the method and the DoF of the robot ordered ascendingly in each method using a distinctive color for each DoF as shown in the legend of the figure. It can be distinguished that, of all the algorithms, *FABRIK* is the one that has its typical values in a lower range than the other methods such as GA, PSO and QPSO whose number of iterations increases as the DoF of the robot increase. However, in the case of the DLS method, it is better appreciated that as the DoF increased, the range of typical values of the number of iterations for this method decreased considerably. It can also be seen that all the methods came to converge for the limit of 1000 iterations, since there are cases where this limit is reached.



5. Box-and-whisker plot of the number of iterations vs methods and DoF

Fig 6 shows the dispersion of the error according to the method and the DoF of the robot ordered ascendingly in each method using a distinctive color for each DoF as shown in the legend of the figure. It can be seen that, of all the methods, the GA is the one with the lowest error, while the proposed method has its typical values just as low as the GA, but it has large outliers of up to approximately a value of 5. On the other hand, it can be seen that for the PSO and QPSO methods, their errors increase as the robot's GDL increases.



6. Box-and-whisker plot of the final RMSE vs methods and DoF

Below, Table 2 shows, as a summary of the experiments carried out with the participating algorithms, the average time,

the final error and the corresponding number of iterations for each method and according to the DoF of the robot.

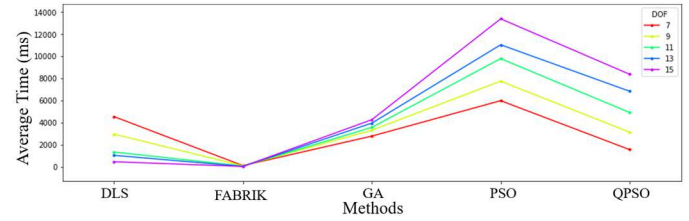
II. SUMMARY OF THE MEASUREMENTS OBTAINED FROM EACH ALGORITHM AND DOF OF THE ROBOT

Method	DoF	Average Time (ms)	Average Final Error	Average Number of iterations
DLS	7	4519.87	3.47230	585.31
	9	2934.65	1.63374	282.83
	11	1320.83	0.59328	118.32
	13	1024.24	0.21585	74.65
	15	443.00	0.02948	28.99
FABRIK	7	96.19	0.08738	48.51
	9	77.56	0.07179	32.43
	11	58.56	0.04020	19.68
	13	25.15	0.02019	7.08
	15	19.23	0.00341	4.24
GA	7	2758.27	0.00122	113.14
	9	3292.70	0.00093	104.92
	11	3545.79	0.00095	106.94
	13	3932.27	0.00088	102.25
	15	4241.40	0.00087	104.26
PSO	7	5962.48	0.10371	708.49
	9	7722.65	0.19860	764.28
	11	9764.63	0.33241	825.85
	13	11013.31	0.49156	838.89
	15	13357.55	0.58096	876.19
QPSO	7	1541.53	0.01023	284.07
	9	3121.81	0.03601	442.14
	11	4896.09	0.09531	568.01
	13	6815.26	0.17189	696.23
	15	8350.98	0.23532	760.05

Now, to better illustrate the interaction between the data shown in the previous table, interaction diagrams were made where the average value of the evaluated parameter of each method is attended according to the DoF of the robot instead of attending to the averages and typical values that were seen in the box and whisker plots shown previously.

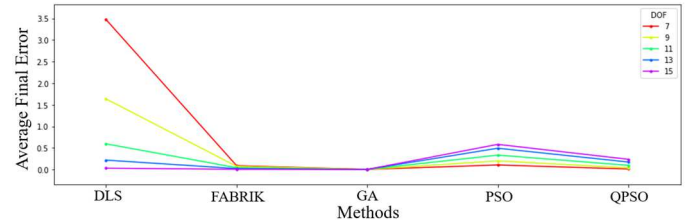
In Fig 7, with respect to the average time invested by the compared algorithms, the FABRIK method is the one that presents the lowest investment of average time made and that this value remains stable even when the GDL of the robot is varied, unlike the AG, PSO and QPSO methods whose average time increases as the DoF increase. However, this graph

confirms that the average DLS time decreased as the DoF increased.



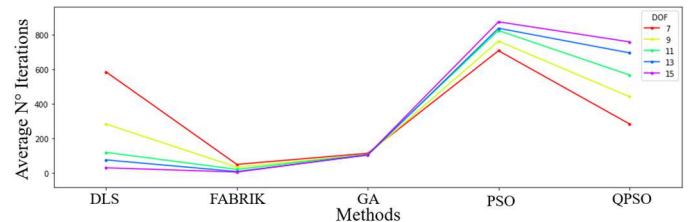
7. Interaction diagram of the convergence time according to the methods and the DoF of the robot

On the other hand, in Figure 8 it can be seen that with respect to the average final error of the compared algorithms, the FABRIK method was in second place compared to the GA, whose average RMSE did not vary significantly as the DoF of the robot increased. However, both the RMSE of the proposed method and the DLS decreased as the DoF increased, but those of the PSO and QPSO increased as more DoF were added.



8. Interaction diagram of the final error according to the methods and the DoF of the robot

Now, in Fig 9 with respect to the average number of iterations invested by the compared algorithms, the FABRIK method is the one that presents the lowest number of average iterations and that this value gradually decreases when the DoF of the robot increases, unlike the PSO and QPSO methods whose average number of iterations increased as the DoF increased. And surprisingly, the DLS method drastically decreases the average number of iterations as the DoFs increased.



9. Interaction diagram of the number of iterations according to the methods and the DoF of the robot

IV. CONCLUSIONS

After carrying out this study, the following conclusions were reached:

- Regarding the time required to solve the inverse kinematics, it was observed that the FABRIK method is the one that invested the least possible time among the algorithms and that, on average, its value did not change significantly as the DoF of the kinematic chains

increased. Whereas the DLS method, contrary to what would be expected, as the DoF increased, the average time required to solve the inverse kinematics decreased considerably. And the other methods based on stochastic behavior of individuals (GA, PSO and QPSO) presented an increase in the average solution time as the DoF increased.

- Regarding the final error of the compared algorithms, the method that presented the best performance in this aspect was the genetic algorithm, since its average value was always below the threshold of 0.001. Secondly, the FABRIK algorithm was found, whose average error decreased as the DoF increased. On the other hand, the average final error of the DLS method also decreased considerably as the DoF were increased; whereas in the PSO and QPSO methods the average final error increased as the DoF increased.
- Regarding the number of iterations, the FABRIK method was the algorithm that had the lowest average numbers among the others, and as the DoF increased, this value gradually decreased. Also, the DLS method decreased the number of average iterations considerably as these degrees were increased, and the genetic algorithm did not change this average value as much as was preserved in the interaction plot. On the other side, the PSO and QPSO presented a significant increase in the number of average iterations as the degrees of freedom increased.

REFERENCES

- Aristidou, A., & Lasenby, J. (2011). FABRIK: A fast, iterative solver for the Inverse Kinematics problem. *Graphical Models*, 73(5), 243–260. <https://doi.org/10.1016/j.gmod.2011.05.003>
- Aristidou, A., Chrysanthou, Y., & Lasenby, J. (2016). Extending FABRIK with model constraints. *Computer Animation and Virtual Worlds*, 27(1), 35–57. <https://doi.org/10.1002/cav.1630>
- Sun, J., Cao, G., Li, W., Liang, Y., & Huang, S. (2017). Analytical inverse kinematic solution using the D-H method for a 6-DOF robot. <https://doi.org/10.1109/urui.2017.7992807>
- Tian, X., Xu, Q., & Zhan, Q. (2021). An analytical inverse kinematics solution with joint limits avoidance of 7-DOF anthropomorphic manipulators without offset. *Journal of the Franklin Institute-engineering and Applied Mathematics*, 358(2), 1252–1272. <https://doi.org/10.1016/j.jfranklin.2020.11.020>
- Bensadoun, R., Gur, S., Blau, N., Shenkar, T., & Wolf, L. (2022). Neural inverse kinematics. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2205.10837>
- Aggogeri, F., Pellegrini, N., Taesi, C., & Tagliani, F. L. (2022). Inverse kinematic solver based on machine learning sequential procedure for robotic applications. *Journal of Physics*, 2234(1), 012007. <https://doi.org/10.1088/1742-6596/2234/1/012007>
- Malik, A., Lischuk, Y., Henderson, T., & Praznica, R. J. (2022). A Deep Reinforcement-Learning approach for inverse Kinematics solution of a high degree of freedom robotic manipulator. *Robotics*, 11(2), 44. <https://doi.org/10.3390/robotics11020044>
- Chiaverini, S. (1993). Estimate of the two smallest singular values of the Jacobian Matrix: Application to damped least-squares inverse kinematics. *Journal of Robotic Systems*, 10(8), 991–1008. <https://doi.org/10.1002/rob.4620100802>
- Drexler, D. A., & Harmati, I. Á. (2012). Joint Constrained Differential Inverse Kinematics algorithm for serial manipulators. *Periodica Polytechnica Electrical Engineering*, 56(4), 95. <https://doi.org/10.3311/ppee.7163>
- Dantam, N. T. (2020). Robust and efficient forward, differential, and inverse kinematics using dual quaternions. *The International Journal of Robotics Research*, 40(10–11), 1087–1105. <https://doi.org/10.1177/0278364920931948>
- Coevoet, E., Escande, A., & Duriez, C. (2017). Optimization-Based inverse model of soft robots with contact handling. *IEEE Robotics and Automation Letters*, 2(3), 1413–1419. <https://doi.org/10.1109/lra.2017.2669367>
- Kennedy, J. F., & Eberhart, R. C. (2002b). Particle swarm optimization. <https://doi.org/10.1109/icnn.1995.488968>
- Alkayyali, M., & Tutunji, T. A. (2019). PSO-based algorithm for inverse kinematics solution of robotic arm manipulators. <https://doi.org/10.1109/rem.2019.8744103>
- Collinsm, T. J., & Shen, W. (2017). Particle Swarm Optimization for high-DOF inverse kinematics. <https://doi.org/10.1109/iccar.2017.7942651>
- Alexis, J., Alejandro, E. M., Arturo, F. S., Gustavo, R. R., Alfredo, E. P., & Vázquez-Castillo, V. (2021). Particle swarm optimization for inverse kinematics solution and trajectory planning of 7-DOF and 8-DOF robot manipulators based on unit quaternion representation. *Istraživanja I Projektovanja Za Privredu*, 19(3), 592–599. <https://doi.org/10.5937/jaes0-30557>
- Xi, M., Wu, X., Sheng, X., Sun, J., & Xu, W. (2016). Improved quantum-behaved particle swarm optimization with local search strategy. *Journal of Algorithms & Computational Technology*, 11(1), 3–12. <https://doi.org/10.1177/1748301816654020>
- Dereli, S., & Köker, R. (2019). A meta-heuristic proposal for inverse kinematics solution of 7-DOF serial robotic manipulator: quantum behaved particle swarm algorithm. *Artificial Intelligence Review*, 53(2), 949–964. <https://doi.org/10.1007/s10462-019-09683-x>
- Momani, S., Abo-Hammour, Z. S., & Alsmadi, O. M. (2016). Solution of Inverse Kinematics Problem using Genetic Algorithms. *Applied Mathematics & Information Sciences*, 10(1), 225–233. <https://doi.org/10.18576/amis/100122>
- Lee, C., & Chang, J. (2021). A Workspace-Analysis-Based genetic algorithm for solving inverse kinematics of a Multi-Fingered anthropomorphic hand. *Applied Sciences*, 11(6), 2668. <https://doi.org/10.3390/app11062668>
- Abdallah, M. M. S., Baziyeed, M. S., Fareh, R., & Rabie, T. (2018). Tracking control for robotic manipulator based on FABRIK algorithm. In *2018 Advances in Science and Engineering Technology International Conferences (ASET)*. <https://doi.org/10.1109/icaset.2018.8376844>
- Tenneti, R., & Sarkar, A. (2019). Implementation of modified FABRIK for robot manipulators. <https://doi.org/10.1145/3352593.3352605>
- Dong, G., Huang, P., Wang, Y., & Li, R. (2022). A modified forward and backward reaching inverse kinematics based incremental control for space manipulators. *Chinese Journal of Aeronautics*, 35(12), 287–295. <https://doi.org/10.1016/j.cja.2021.08.014>
- Santos, M. F., Molina, L., Carvalho, E. a. N., Freire, E. O., Carvalho, J. C. A., & Santos, P. C. (2021). FABRIK-R: an extension developed based on FABRIK for robotics manipulators. *IEEE Access*, 9, 53423–53435. <https://doi.org/10.1109/access.2021.3070693>
- Moreno, L. a. O., & Alcantara, J. H. A. (2022). An adaptation of FABRIK algorithm for serial robot's inverse kinematics. <https://doi.org/10.1109/icev56253.2022.9959663>
- Santos, P. C., Freire, R., Carvalho, E. a. N., Molina, L., & Freire, E. O. (2020). M-FABRIK: A new inverse kinematics approach to mobile manipulator robots based on FABRIK. *IEEE Access*, 8, 208836–208849. <https://doi.org/10.1109/access.2020.3038424>
- Tao, S., Tao, H., & Yang, Y. (2021). Extending FABRIK with Obstacle Avoidance for Solving the Inverse Kinematics Problem. *Journal of Robotics*, 2021, 1–10. <https://doi.org/10.1155/2021/5568702>
- Sengupta, Saptarshi & Basak, Sanchita & Alan Peters II, Richard. (2018). Particle Swarm Optimization: A survey of historical and recent developments with hybridization perspectives. *10.3390/make1010010*